

Arquitectura Híbrida de Datos

EQUIPO DE TRABAJO: E01

Camilo José Mora Rodríguez 0000394950

Carlos Alberto Zambrano Braydi 0000395349

Alejandro Miguel López Castañeda 0000385228

Carlos Arturo Salazar Castañeda 0000393238

Contexto y Objetivos

Contexto

- Plataforma de eCommerce con crecimiento constante de productos y transacciones.
- Necesidad de manejar información estructurada y flexible.
- Requerimiento de escalabilidad y buen desempeño.

Objetivos

- Optimizar el almacenamiento según el tipo de dato.
- Garantizar consistencia en procesos de compra.
- Mejorar la flexibilidad del catálogo de productos.
- Diseñar una arquitectura preparada para el crecimiento

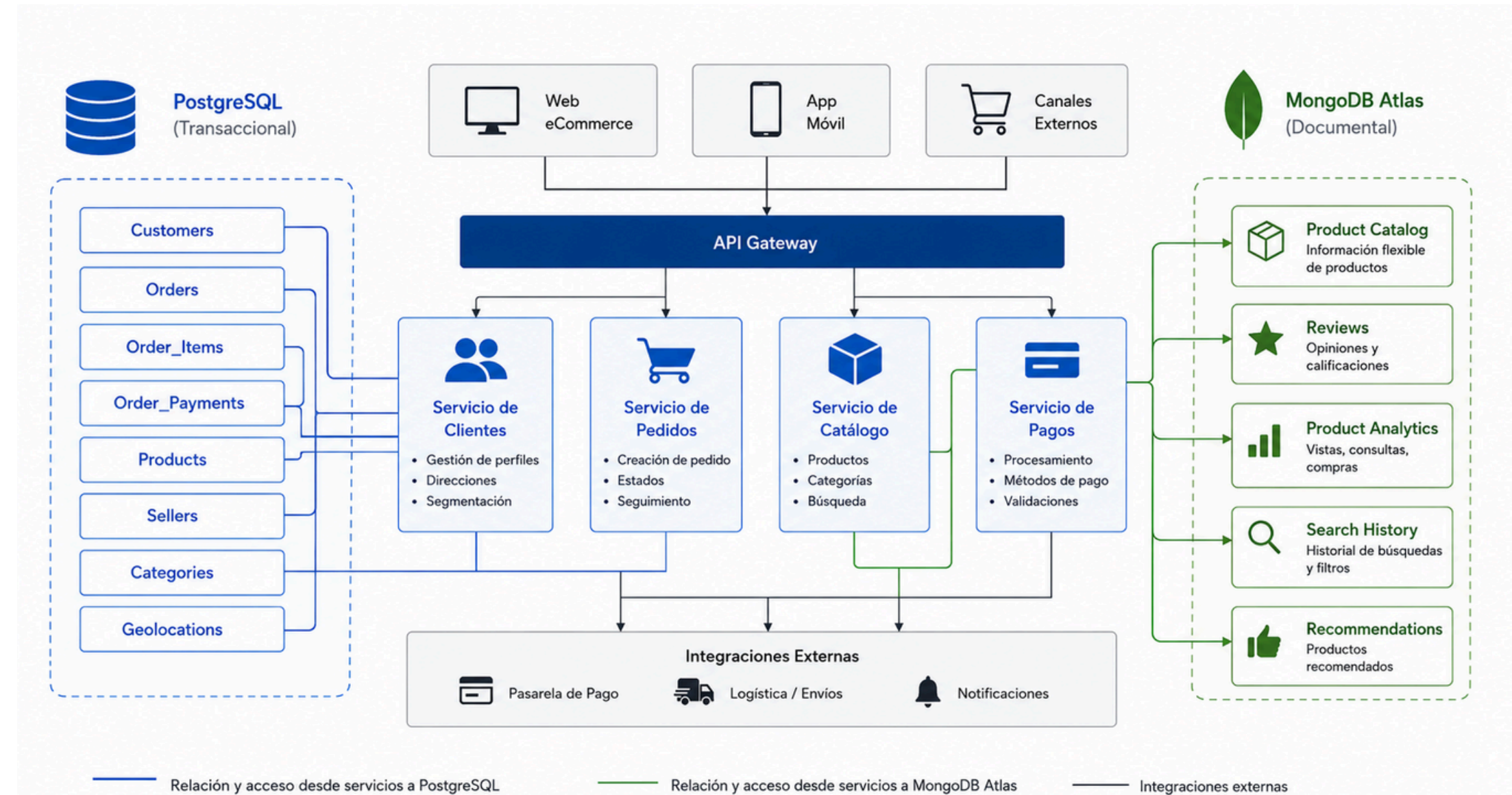
Arquitectura Híbrida Implementada

Componentes principales

- PostgreSQL para la gestión transaccional.
- MongoDB Atlas para catálogo y datos documentales.
- API central para integración entre servicios.
- Separación de responsabilidades por tipo de información.

Beneficios obtenidos

- Mayor flexibilidad.
- Mejor organización de los datos.
- Escalabilidad independiente por componente.



Decisiones y Justificación

Decisiones adoptadas

- Uso de PostgreSQL para órdenes, pagos y clientes.
- Uso de MongoDB para productos, reseñas y búsquedas.
- Implementación de Polyglot Persistence.
- Integración mediante identificadores compartidos.



Justificación

- Cada motor se utiliza donde aporta mayor valor.
- Menor complejidad en consultas especializadas.
- Mejor aprovechamiento de recursos.



Evaluación de Rendimiento

La evaluación de rendimiento se realizó sobre los dos motores de la arquitectura híbrida de forma independiente, utilizando las herramientas nativas de cada sistema, EXPLAIN (ANALYZE, BUFFERS) para PostgreSQL en Supabase y `.explain('executionStats')` para MongoDB en Atlas, en ambos casos se midió el estado antes y después de cada optimización para obtener evidencia cuantificable de las mejoras.

Hallazgos principales

- Consultas de catálogo más eficientes en MongoDB.
- Operaciones transaccionales estables en PostgreSQL.
- Menor impacto entre cargas transaccionales y documentales.
- Mejor distribución de procesamiento entre motores.



PostgreSQL vs. MongoDB

Módulo / Entidad	Motor	Naturaleza del dato	Razón de la asignación
customers, sellers, orders, order_items, order_payments, categories	PostgreSQL	Estructurado, integridad estricta	ACID multi-tabla, FK, montos NUMERIC sin pérdida
products (catálogo rico)	MongoDB	Semi-estructurado, atributos variables	Attribute/Embedding Pattern; specs por categoría sin tablas EAV
order_reviews	MongoDB	Alto volumen, lectura analítica	Referenced Pattern; escala por sharding sin tocar la app
geolocation	MongoDB	Geoespacial (GeoJSON)	Índices 2dsphere para consultas por proximidad
eventos analíticos / logs / search	MongoDB	Append-only, alto throughput	Disponibilidad sobre consistencia; índices TTL y de texto

Análisis del Teorema CAP

PostgreSQL

- Excelente integridad de datos.
- Soporte robusto para transacciones.
- Ideal para procesos críticos del negocio.

MongoDB Atlas

- Alta flexibilidad de estructura.
- Escalabilidad horizontal.
- Optimizado para catálogos dinámicos y consultas documentales.

Conclusión: Tecnologías complementarias, no competidoras.

Lecciones y Desafíos

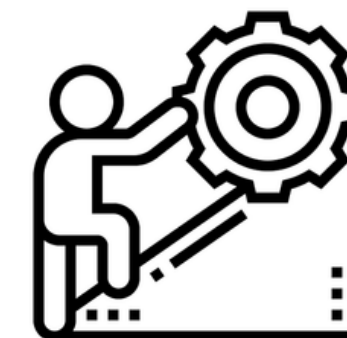
Lecciones

- No toda la información debe almacenarse en el mismo motor.
- El diseño de datos impacta directamente el rendimiento.
- La escalabilidad debe considerarse desde el inicio.



Desafíos

- Definir la distribución adecuada de la información.
- Mantener consistencia entre ambos entornos.
- Diseñar integraciones simples y mantenibles.



Recomendaciones Estratégicas

Incorporar mecanismos de caché para consultas frecuentes.



Implementar monitoreo y observabilidad continua.



Automatizar respaldos y recuperación.



Evolucionar hacia una arquitectura orientada a eventos.

Conclusiones

Conclusiones principales

- La arquitectura híbrida cumplió los objetivos planteados.
- PostgreSQL garantiza confiabilidad en las operaciones críticas.
- MongoDB Atlas aporta flexibilidad para información dinámica.
- La combinación mejora rendimiento, escalabilidad y mantenibilidad.
- La solución queda preparada para futuras necesidades del negocio.



Universidad de
La Sabana

Gracias